POLITEKNIK MANUFAKTUR BANDUNG

Jalan Kanayakan 21 Tlp/Fax. +62 (022) 7312564 Bandung 40275
Telp. (022)2500241 Faks. (022)2502649 https://polman-bandung.ac.id/

AE

LAPORAN PRAKTIKUM MACHINE LEARNING (MINGGU - 2)

Nama : **Muhammad Miftah Rizqi**
NIM : **224443061**
Kelas : **3AEC3**

PROGRAM STUDI D4 TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
JURUSAN OTOMASI MANUFAKTUR DAN MEKATRONIKA
POLITEKNIK MANUFAKTUR BANDUNG
2026

DAFTAR ISI

DAFTAR ISI ..... **i**
BAB I **1**
1.1 Latar Belakang **1**
1.2 Rumusan Masalah **2**
1.3 Tujuan Praktikum **3**
1.4 Manfaat Praktikum **3**
1.3.1 Manfaat Teoritis 3
1.3.2 Manfaat Praktis 4
1.3.3 Manfaat bagi Pengembangan Kompetensi Mahasiswa 4
BAB II **5**
2.1 Machine Learning **5**
2.1.1 Supervised Learning 5
2.1.2 Unsupervised Learning 5
2.1.3 Reinforcement Learning 6
2.2 Supervised Learning **6**
2.3 Klasifikasi **8**
2.4 Algoritma **8**
2.4.1 Decision Tree 8
2.4.2 K-Nearest Neighbors 8
2.4.3 Naïve Bayes 9
2.4.4 Logistic Regression 9
2.4.5 Random Forest 9
2.5 Evaluasi Model **10**
2.5.1 Confusion Matrix 10
2.5.2 Accuracy 11
2.5.3 Precision 11
2.5.4 Recall 11
2.5.5 F-1 Score 11
2.5.6 AUC-ROC 12
2.6 Penanganan Missing Value Sederhana **12**
2.6.1 *Synthetic Minority Over-sampling Technique* (SMOTE) 12
2.6.2 *Class Weight* 13

2.6.3 Pemilihan Strategi Penanganan 13
BAB III 15
3.1 Alur Kerja (*Flowchart*) 15
3.2 Dataset 15
3.3 Tools dan Library 16
BAB IV 17
4.1 Eksplorasi Data (EDA) 17
4.1.1 Informasi Umum Dataset 17
4.1.2 Pengecakan *Missing Value* dan Duplikat 17
4.1.3 Distribusi Target 18
4.1.4 Visualisasi Distribusi Fitur Numerik 19
4.1.5 Heatmap Korelasi 20
4.1.6 Analisis Fitur vs Target 21
4.2 Preprocessing 22
4.2.1 Feature Engineering 22
4.2.2 *Split Data* 23
4.2.3 *Feature Scaling* 24
4.3 Modeling 25
4.4 Evaluasi Model 28
4.4.1 Tabel Perbandingan Metrik 28
4.4.2 Confusion Matrix Model Terbaik 29
4.4.3 Classification Report 30
4.4.2 ROC Curve 32
4.5 Pembahasan 34
4.5.1 Analisis Model Terbaik 34
4.5.2 Analisis Overfitting 34
4.5.3 Fitur Paling Berpengaruh 34
4.5.4 Implikasi untuk Quality Control 34
4.5.5 Keterbatasan 35
BAB V 36
5.1 Kesimpulan 36
5.2 Saran 36
Link Github Tugas mandiri: 36

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital dan kecerdasan buatan telah memberikan pengaruh yang signifikan terhadap berbagai sektor industri, termasuk industri manufaktur. Salah satu aspek penting dalam industri manufaktur adalah pengendalian kualitas (quality control), yang bertujuan untuk memastikan bahwa produk yang dihasilkan sesuai dengan standar dan spesifikasi yang telah ditentukan. Kualitas produk menjadi faktor penting karena berpengaruh terhadap kepuasan pelanggan, efisiensi biaya produksi, daya saing perusahaan, serta keselamatan pengguna.

Dalam proses produksi, produk cacat (defect) merupakan salah satu permasalahan yang dapat menimbulkan kerugian bagi perusahaan. Cacat produk dapat berupa ketidaksesuaian dimensi, kerusakan permukaan, cacat struktural, kegagalan fungsi, maupun kesalahan perakitan. Pendekatan pengendalian kualitas secara konvensional umumnya dilakukan setelah produk selesai diproduksi. Pendekatan tersebut bersifat reaktif karena cacat baru diketahui setelah proses produksi berlangsung, sehingga berpotensi meningkatkan biaya pemeriksaan, perbaikan, dan pemborosan material.

Seiring dengan perkembangan teknologi Machine Learning, pengendalian kualitas dapat dikembangkan melalui pendekatan prediktif (predictive quality). Pendekatan ini memungkinkan sistem mempelajari pola dari data produksi untuk memperkirakan kemungkinan terjadinya cacat sebelum produk selesai diproduksi. Data yang digunakan dapat berasal dari parameter proses produksi, kondisi mesin, sensor, lingkungan kerja, serta aktivitas pemeliharaan mesin. Dengan memanfaatkan informasi tersebut, perusahaan dapat mengidentifikasi faktor-faktor yang berkaitan dengan cacat produk dan mengambil tindakan pencegahan secara lebih dini.

Salah satu metode Machine Learning yang dapat digunakan untuk permasalahan tersebut adalah Supervised Learning. Metode ini merupakan pendekatan pembelajaran mesin yang menggunakan data berlabel untuk mempelajari hubungan antara variabel masukan (fitur) dan variabel keluaran (label). Dalam klasifikasi cacat produk manufaktur, model dilatih menggunakan data yang memiliki informasi mengenai karakteristik produksi

dan status kualitas produk. Model kemudian digunakan untuk mengklasifikasikan produk ke dalam kategori yang telah ditentukan, seperti High Defect dan Low Defect.

Penerapan Supervised Learning memerlukan beberapa tahapan, mulai dari eksplorasi data (Exploratory Data Analysis atau EDA), prapemrosesan data (data preprocessing), pembentukan model klasifikasi, hingga evaluasi kinerja model. Algoritma yang dapat digunakan antara lain Decision Tree, K-Nearest Neighbors (KNN), Naive Bayes, Logistic Regression, Random Forest, dan XGBoost. Setiap algoritma memiliki karakteristik yang berbeda sehingga pemilihan model perlu mempertimbangkan karakteristik data dan kebutuhan klasifikasi.

Selain pemilihan algoritma, permasalahan ketidakseimbangan kelas (class imbalance) juga menjadi tantangan dalam klasifikasi cacat produk. Jumlah produk cacat dapat jauh lebih sedikit dibandingkan produk yang tidak cacat, sehingga model berpotensi menghasilkan akurasi tinggi tetapi kurang mampu mendeteksi produk cacat. Oleh karena itu, diperlukan teknik penanganan ketidakseimbangan kelas serta metrik evaluasi yang sesuai, seperti precision, recall, F1-score, dan AUC-ROC. Khusus dalam deteksi cacat, recall menjadi metrik penting karena berkaitan dengan kemampuan model mengenali produk yang benar-benar memiliki risiko cacat.

Berdasarkan hal tersebut, praktikum ini dilaksanakan untuk mempelajari penerapan Supervised Learning dalam klasifikasi cacat produk manufaktur melalui proses pengolahan data, pembangunan model, dan evaluasi kinerja klasifikasi. Praktikum ini diharapkan dapat memberikan pemahaman mengenai pemanfaatan Machine Learning sebagai salah satu pendekatan untuk mendukung pengendalian kualitas yang lebih efektif, proaktif, dan berbasis data dalam industri manufaktur.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam praktikum ini adalah sebagai berikut.

1. Apa yang dimaksud dengan *Supervised Learning* dan bagaimana penerapannya dalam klasifikasi cacat produk manufaktur?
2. Bagaimana tahapan eksplorasi dan prapemrosesan data dilakukan untuk mempersiapkan data manufaktur sebelum digunakan dalam proses pemodelan?

3. Bagaimana algoritma klasifikasi *Machine Learning* digunakan untuk memprediksi status cacat produk berdasarkan fitur-fitur yang tersedia?
4. Bagaimana ketidakseimbangan kelas dalam data manufaktur dapat memengaruhi kinerja model dan bagaimana cara menanganinya?
5. Bagaimana kinerja model klasifikasi dievaluasi menggunakan metrik yang sesuai, seperti *precision*, *recall*, *F1-score*, dan *AUC-ROC*?
6. Bagaimana hasil klasifikasi dapat dimanfaatkan untuk mendukung proses pengendalian kualitas produk manufaktur?

1.3 Tujuan Praktikum

Praktikum ini bertujuan untuk:

1. Memahami konsep dasar *Supervised Learning* dan penerapannya dalam klasifikasi cacat produk manufaktur.
2. Memahami dan menerapkan tahapan eksplorasi serta prapemrosesan data sebelum proses pembangunan model *Machine Learning*.
3. Mempelajari penerapan algoritma klasifikasi untuk memprediksi status cacat produk berdasarkan data dan fitur yang tersedia.
4. Memahami permasalahan *class imbalance* serta teknik penanganannya dalam klasifikasi produk manufaktur.
5. Mempelajari proses evaluasi kinerja model menggunakan metrik klasifikasi yang sesuai dengan karakteristik data dan kebutuhan pengendalian kualitas.
6. Memahami pemanfaatan hasil prediksi model sebagai informasi pendukung untuk mengidentifikasi risiko cacat dan meningkatkan efektivitas pengendalian kualitas produk.

1.4 Manfaat Praktikum

Pelaksanaan praktikum ini diharapkan memberikan manfaat sebagai berikut.

1.3.1 Manfaat Teoritis

Praktikum ini dapat menambah pemahaman mengenai konsep *Machine Learning*, khususnya *Supervised Learning* untuk permasalahan klasifikasi. Selain itu, praktikum ini membantu memperdalam pemahaman mengenai tahapan pengolahan data, algoritma klasifikasi, penanganan ketidakseimbangan kelas, serta evaluasi kinerja model dalam konteks pengendalian kualitas produk manufaktur.

1.3.2 Manfaat Praktis

Praktikum ini memberikan pengalaman dalam mengolah dan menganalisis data manufaktur untuk membangun model klasifikasi. Melalui kegiatan tersebut, mahasiswa dapat mempelajari cara menyiapkan data, memilih algoritma, melatih model, dan mengevaluasi hasil prediksi berdasarkan metrik yang relevan. Pengetahuan ini dapat menjadi dasar untuk mengembangkan solusi berbasis data dalam mengidentifikasi potensi cacat produk.

1.3.3 Manfaat bagi Pengembangan Kompetensi Mahasiswa

Praktikum ini dapat meningkatkan keterampilan mahasiswa dalam menerapkan konsep *Machine Learning* untuk menyelesaikan permasalahan yang relevan dengan kebutuhan industri. Mahasiswa juga diharapkan mampu mengembangkan kemampuan berpikir analitis, menginterpretasikan hasil pemodelan, serta memahami pentingnya penggunaan data dalam pengambilan keputusan. Dengan demikian, praktikum ini dapat menjadi bekal dalam menghadapi penerapan teknologi digital dan kecerdasan buatan pada sektor manufaktur.

BAB II
LANDASAN TEORI

2.1 Machine Learning

Machine Learning merupakan bagian dari kecerdasan buatan (*Artificial Intelligence*) yang memungkinkan komputer mempelajari pola dari data untuk menghasilkan prediksi atau keputusan tanpa harus diprogram menggunakan aturan khusus untuk setiap kondisi. Dalam industri manufaktur, *Machine Learning* dapat digunakan untuk menganalisis data produksi, mengenali pola yang berkaitan dengan kualitas produk, serta memperkirakan kemungkinan terjadinya cacat (*defect*).

Penerapan *Machine Learning* dalam pengendalian kualitas memberikan peluang bagi perusahaan untuk beralih dari pendekatan pemeriksaan yang bersifat reaktif menuju pendekatan prediktif. Melalui analisis data historis dan data proses produksi, model dapat mempelajari hubungan antara karakteristik proses dan kondisi kualitas produk. Hasil analisis tersebut dapat digunakan sebagai informasi pendukung dalam menentukan tindakan perbaikan atau pencegahan.

Secara umum, *Machine Learning* dapat dikelompokkan menjadi tiga jenis utama, yaitu *Supervised Learning*, *Unsupervised Learning*, dan *Reinforcement Learning*.

2.1.1 Supervised Learning

Supervised Learning merupakan paradigma Machine Learning yang menggunakan data berlabel sebagai dasar pembelajaran model. Data tersebut terdiri atas pasangan fitur atau masukan ((x)) dan label atau keluaran ((y)). Model mempelajari hubungan antara fitur dan label untuk menghasilkan prediksi terhadap data baru yang belum pernah dipelajari sebelumnya.

Dalam konteks manufaktur, fitur dapat berupa suhu mesin, tekanan, kecepatan produksi, volume produksi, atau skor kualitas. Sementara itu, label dapat berupa status kualitas produk, seperti High Defect dan Low Defect. Model kemudian mempelajari pola dari data tersebut untuk memperkirakan kategori produk berdasarkan karakteristik proses produksinya

2.1.2 Unsupervised Learning

Unsupervised Learning merupakan metode pembelajaran mesin yang bekerja menggunakan data tanpa label target. Model berusaha menemukan pola,

struktur, kelompok, atau hubungan tertentu berdasarkan karakteristik data yang tersedia.

Dalam konteks manufaktur, metode ini dapat digunakan untuk mengelompokkan produk berdasarkan kemiripan karakteristik proses, menemukan pola operasi mesin, atau mengidentifikasi data produksi yang menyimpang dari pola umum. Salah satu teknik yang sering digunakan adalah *clustering*, yaitu pengelompokan data berdasarkan kemiripan karakteristiknya.

2.1.3 Reinforcement Learning

Reinforcement Learning merupakan metode pembelajaran mesin yang melibatkan agen yang belajar mengambil tindakan melalui interaksi dengan lingkungan. Setiap tindakan dapat menghasilkan *reward* atau *penalty* yang digunakan untuk memperbaiki strategi pengambilan keputusan.

Dalam konteks manufaktur, pendekatan ini berpotensi digunakan untuk mengoptimalkan pengaturan parameter mesin, penjadwalan produksi, atau strategi pengendalian proses. Agen belajar memilih tindakan yang dapat memberikan hasil terbaik berdasarkan tujuan dan umpan balik yang diterima dari lingkungan.

2.2 Supervised Learning

Supervised Learning adalah pendekatan *Machine Learning* yang memanfaatkan data berlabel untuk membangun model prediksi. Pada metode ini, model dilatih menggunakan pasangan data masukan dan keluaran yang telah diketahui. Selama proses pelatihan, model berusaha mempelajari hubungan antara fitur dan label agar dapat menghasilkan prediksi pada data yang belum pernah dilihat sebelumnya.

Dalam klasifikasi cacat produk manufaktur, data pelatihan dapat berisi informasi mengenai parameter proses produksi, kondisi mesin, dan karakteristik produk. Data tersebut disertai label yang menunjukkan status kualitas produk, sehingga model dapat mempelajari karakteristik yang berhubungan dengan kategori produk cacat atau tidak cacat.

2.2.1 Fitur (*Features*)

Fitur merupakan variabel masukan yang digunakan oleh model untuk mempelajari pola dan menghasilkan prediksi. Fitur menggambarkan karakteristik atau kondisi dari objek yang dianalisis. Pada data manufaktur, fitur yang relevan dapat mencakup:

1. Parameter proses produksi, seperti suhu, tekanan, kecepatan produksi, dan waktu proses.
2. Kondisi mesin, seperti getaran, temperatur mesin, atau indikator kondisi peralatan.
3. Faktor lingkungan, seperti temperatur dan kelembapan lingkungan kerja.
4. Informasi proses, seperti jenis material, identitas lini produksi, atau parameter pemeliharaan yang tersedia dalam dataset.

Pemilihan fitur perlu disesuaikan dengan data yang tersedia dan konteks proses produksi. Fitur yang relevan diharapkan dapat membantu model mengenali pola yang berkaitan dengan kualitas produk.

2.2.2 Label

Label merupakan variabel target yang ingin diprediksi oleh model. Dalam pembelajaran terawasi, label pada data pelatihan harus diketahui agar model dapat membandingkan hasil prediksi dengan kondisi sebenarnya.

Dalam klasifikasi cacat produk, label dapat berupa dua kategori, yaitu *High Defect* dengan nilai 1 dan *Low Defect* dengan nilai 0, sesuai dengan skema klasifikasi yang digunakan dalam materi praktikum. Label tersebut menjadi acuan bagi model untuk mempelajari perbedaan pola pada setiap kategori.

2.2.3 Tahapan Penerapan

Penerapan *Supervised Learning* umumnya meliputi beberapa tahapan berikut.

1. **Definisi masalah**, yaitu menentukan tujuan prediksi dan kategori target yang ingin dibedakan.
2. **Pengumpulan data**, yaitu memperoleh data produksi yang memiliki fitur dan label.
3. **Eksplorasi data (EDA)**, yaitu memahami struktur, distribusi, hubungan antarvariabel, dan kemungkinan permasalahan data.
4. **Prapemrosesan data**, yaitu menangani nilai yang hilang, data tidak konsisten, serta menyiapkan fitur agar dapat digunakan oleh algoritma.
5. **Rekayasa fitur (*feature engineering*),** yaitu membentuk atau memilih fitur yang dapat membantu proses pemodelan.
6. **Pembangunan model**, yaitu melatih algoritma menggunakan data pelatihan.

7. **Evaluasi dan penyetelan model,** yaitu menilai performa model serta menyesuaikan parameter untuk memperoleh hasil yang lebih baik.
8. **Interpretasi dan pemantauan**, yaitu memahami hasil model dan memantau kinerjanya ketika digunakan pada data produksi yang baru.

2.3 Klasifikasi

Klasifikasi merupakan salah satu jenis permasalahan dalam Supervised Learning yang bertujuan memprediksi kategori atau kelas dari suatu data berdasarkan fitur yang tersedia. Model klasifikasi mempelajari pola dari data berlabel untuk menentukan kelas yang paling sesuai bagi suatu observasi baru.

Dalam industri manufaktur, klasifikasi dapat digunakan untuk menentukan apakah suatu produk termasuk kategori cacat atau tidak cacat, mengidentifikasi jenis kerusakan, atau mengelompokkan produk berdasarkan tingkat risiko kualitas. Dalam praktikum ini, klasifikasi difokuskan pada penentuan status cacat produk berdasarkan label yang tersedia pada dataset.

2.4 Algoritma

2.4.1 Decision Tree

Decision Tree atau pohon keputusan merupakan algoritma klasifikasi yang membentuk struktur menyerupai pohon untuk menentukan kelas suatu data. Struktur tersebut terdiri atas simpul keputusan (*decision node*), cabang yang mewakili hasil pengujian fitur, dan simpul daun (*leaf node*) yang menunjukkan prediksi kelas.

Model bekerja dengan membagi data secara bertahap berdasarkan fitur yang dianggap paling efektif untuk memisahkan kelas. Sebagai contoh, model dapat membentuk aturan berdasarkan suhu proses atau tekanan mesin untuk membedakan produk yang memiliki risiko cacat tinggi dan rendah. Salah satu kelebihan *Decision Tree* adalah hasilnya relatif mudah diinterpretasikan karena dapat ditampilkan sebagai rangkaian aturan keputusan.

2.4.2 K-Nearest Neighbors

K-Nearest Neighbors (KNN) merupakan algoritma klasifikasi yang menentukan kelas suatu data berdasarkan kelas dari sejumlah tetangga terdekatnya. Algoritma ini termasuk metode berbasis kedekatan karena menggunakan ukuran jarak untuk membandingkan suatu observasi dengan data pelatihan.

Ketika sebuah data baru akan diklasifikasikan, KNN menghitung jaraknya terhadap data pelatihan, memilih sebanyak (K) tetangga terdekat, lalu menentukan kelas berdasarkan suara mayoritas tetangga tersebut. Sebagai contoh, produk baru dapat diklasifikasikan sebagai *High Defect* apabila sebagian besar tetangga terdekatnya memiliki label tersebut.

2.4.3 Naïve Bayes

Naive Bayes merupakan algoritma klasifikasi probabilistik yang menggunakan Teorema Bayes untuk memperkirakan peluang suatu data termasuk ke dalam kelas tertentu berdasarkan fitur yang diamati. Algoritma ini menghitung probabilitas setiap kelas, kemudian memilih kelas dengan probabilitas posterior tertinggi berdasarkan model yang digunakan.

2.4.4 Logistic Regression

Logistic Regression merupakan algoritma yang digunakan untuk memperkirakan probabilitas suatu observasi termasuk ke dalam kelas tertentu. Meskipun mengandung kata *regression*, algoritma ini umum digunakan untuk klasifikasi, terutama klasifikasi biner.

Pada klasifikasi cacat produk, *Logistic Regression* dapat memperkirakan probabilitas bahwa suatu produk termasuk kategori *High Defect*. Probabilitas tersebut kemudian dibandingkan dengan ambang keputusan (*threshold*) untuk menentukan kelas prediksi.

2.4.5 Random Forest

Random Forest merupakan algoritma *ensemble learning* yang menggabungkan prediksi dari banyak pohon keputusan untuk menghasilkan prediksi akhir. Pendekatan ini bertujuan meningkatkan kestabilan model dan mengurangi ketergantungan terhadap satu pohon keputusan.

Dalam klasifikasi, setiap pohon menghasilkan prediksi kelas, kemudian hasil dari seluruh pohon digabungkan, umumnya melalui pemungutan suara mayoritas. Random Forest dapat mempelajari hubungan yang kompleks antara fitur proses produksi dan status cacat produk.

2.5 Evaluasi Model

Evaluasi model merupakan tahap untuk mengukur kemampuan model klasifikasi dalam memprediksi kelas pada data yang digunakan untuk pengujian. Evaluasi yang baik perlu mempertimbangkan tujuan penggunaan model, karakteristik distribusi kelas, dan konsekuensi kesalahan prediksi.

Pada klasifikasi cacat produk manufaktur, kesalahan dalam mendeteksi produk yang benar-benar cacat dapat menimbulkan risiko kualitas. Oleh karena itu, penilaian model sebaiknya tidak hanya mengandalkan satu metrik, melainkan menggunakan beberapa metrik yang saling melengkapi.

2.5.1 Confusion Matrix

Confusion Matrix merupakan tabel yang membandingkan kelas aktual dengan kelas hasil prediksi model. Pada klasifikasi biner, terdapat empat kemungkinan hasil:

Kondisi	**Definisi**	**Contoh dalam manufaktur**
True Positive (TP)	Produk positif diprediksi positif	Produk *High Defect* terdeteksi sebagai *High Defect*
True Negative (TN)	Produk negatif diprediksi negatif	Produk *Low Defect* diprediksi sebagai *Low Defect*
False Positive (FP)	Produk negatif diprediksi positif	Produk *Low Defect* ditandai sebagai *High Defect*
False Negative (FN)	Produk positif diprediksi negatif	Produk *High Defect* diprediksi sebagai *Low Defect*

Dalam deteksi cacat, *False Negative* dapat menjadi kesalahan yang penting karena produk yang seharusnya terdeteksi berisiko cacat justru diklasifikasikan sebagai kelas berisiko rendah. Sementara itu, *False Positive* dapat menyebabkan pemeriksaan tambahan yang sebenarnya tidak diperlukan. Besarnya konsekuensi setiap kesalahan bergantung pada jenis produk dan standar kualitas yang diterapkan.

2.5.2 Accuracy

Accuracy adalah proporsi seluruh prediksi yang benar terhadap jumlah seluruh observasi.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Metrik ini memberikan gambaran umum mengenai ketepatan prediksi model. Namun, pada data dengan *class imbalance*, accuracy dapat terlihat tinggi meskipun model gagal mengenali sebagian besar produk cacat. Oleh karena itu, accuracy perlu ditafsirkan bersama metrik lainnya.

2.5.3 Precision

Precision menunjukkan proporsi prediksi positif yang benar-benar merupakan kelas positif.

$$Precision = \frac{TP}{TP + FP}$$

Precision yang tinggi menunjukkan bahwa sebagian besar produk yang ditandai model sebagai kelas positif memang termasuk kelas positif pada data aktual. Dalam manufaktur, metrik ini berguna untuk menilai seberapa sering produk yang ditandai berisiko cacat benar-benar termasuk kategori tersebut.

2.5.4 Recall

Recall menunjukkan proporsi seluruh kasus positif aktual yang berhasil dikenali oleh model.

$$Recall = \frac{TP}{TP + FN}$$

Recall yang tinggi menunjukkan bahwa model mampu menemukan sebagian besar produk yang benar-benar termasuk kelas positif. Dalam deteksi cacat, recall sering menjadi perhatian utama karena nilai recall yang rendah berarti banyak produk yang berisiko cacat tidak terdeteksi.

2.5.5 F-1 Score

F1-score merupakan rata-rata harmonik antara precision dan recall. Metrik ini memberikan ukuran gabungan yang mempertimbangkan kedua aspek tersebut.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1-score tinggi menunjukkan keseimbangan yang baik antara precision dan recall. Metrik ini dapat berguna ketika distribusi kelas tidak seimbang dan evaluasi perlu mempertimbangkan kemampuan model menemukan kelas positif sekaligus membatasi kesalahan positif.

2.5.6 AUC-ROC

ROC (*Receiver Operating Characteristic*) merupakan kurva yang menggambarkan hubungan antara *True Positive Rate* (TPR) dan *False Positive Rate* (FPR) pada berbagai ambang keputusan.

AUC (*Area Under the Curve*) adalah luas daerah di bawah kurva ROC. Nilai AUC yang lebih tinggi umumnya menunjukkan kemampuan model yang lebih baik dalam membedakan kelas positif dan negatif berdasarkan skor prediksinya.

AUC-ROC bermanfaat untuk membandingkan kemampuan diskriminasi beberapa model pada berbagai ambang keputusan. Akan tetapi, ketika kelas positif sangat jarang, AUC-ROC perlu ditafsirkan bersama precision, recall, dan metrik lain karena kurva ROC dapat terlihat baik meskipun ketepatan prediksi positif masih rendah.

2.6 Penanganan Missing Value Sederhana

Class imbalance merupakan kondisi ketika jumlah observasi pada setiap kelas dalam dataset berbeda secara signifikan. Dalam konteks manufaktur, produk cacat sering kali lebih jarang ditemukan dibandingkan produk yang memenuhi standar kualitas. Kondisi ini dapat terjadi karena proses produksi yang terkendali menghasilkan sebagian besar produk sesuai spesifikasi, sedangkan produk cacat hanya muncul pada kondisi atau kejadian tertentu.

Ketidakseimbangan kelas dapat memengaruhi proses pelatihan model. Model dapat cenderung memprediksi kelas mayoritas karena kelas tersebut mendominasi data pelatihan. Akibatnya, model mungkin memperoleh accuracy yang tinggi tetapi tidak mampu mendeteksi produk cacat secara memadai. Hal ini menjadi masalah apabila tujuan utama sistem adalah mengenali sebanyak mungkin produk yang benar-benar cacat.

2.6.1 *Synthetic Minority Over-sampling Technique* (SMOTE)

Pendekatan pertama adalah SMOTE merupakan teknik *oversampling* yang digunakan untuk menambah jumlah sampel pada kelas minoritas dengan membentuk

sampel sintetis. Berbeda dengan sekadar menggandakan observasi yang sudah ada, SMOTE menghasilkan sampel baru melalui interpolasi antara suatu sampel kelas minoritas dan beberapa tetangga terdekatnya dari kelas yang sama.

SMOTE dapat membantu model mempelajari pola kelas minoritas karena jumlah sampel kelas tersebut bertambah. Namun, teknik ini tidak selalu meningkatkan kinerja model dan dapat menghasilkan sampel yang kurang representatif jika data memiliki noise atau distribusi kelas yang saling tumpang tindih.

Penerapan SMOTE perlu dilakukan hanya pada data pelatihan, bukan sebelum pembagian data secara keseluruhan. Pada validasi silang, oversampling juga harus dilakukan di dalam setiap bagian pelatihan. Langkah tersebut penting untuk menghindari *data leakage*, yaitu masuknya informasi dari data validasi atau pengujian ke dalam proses pelatihan.

2.6.2 *Class Weight*

Class weight merupakan pendekatan yang memberikan bobot berbeda pada kelas ketika model menghitung kesalahan prediksi selama pelatihan. Kelas minoritas yang dianggap lebih penting dapat diberikan bobot lebih tinggi sehingga kesalahan pada kelas tersebut memperoleh penalti yang lebih besar.

Sebagai contoh, apabila produk *High Defect* lebih jarang ditemukan daripada produk *Low Defect*, bobot kelas positif dapat ditingkatkan agar model lebih memperhatikan kesalahan dalam mengenali produk berisiko cacat. Pengaturan ini tersedia pada beberapa algoritma melalui parameter seperti class_weight, bergantung pada implementasi yang digunakan.

Keuntungan pendekatan ini adalah tidak harus menambah sampel sintetis karena penyesuaian dilakukan melalui pembobotan kesalahan. Namun, pemberian bobot yang terlalu tinggi dapat meningkatkan recall sekaligus menambah false positive, sehingga parameter perlu dievaluasi menggunakan metrik yang sesuai.

2.6.3 Pemilihan Strategi Penanganan

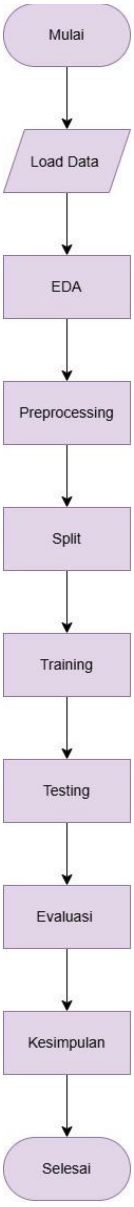
SMOTE dan *class weight* merupakan dua strategi yang dapat dipertimbangkan untuk mengatasi *class imbalance*. Pemilihan strategi bergantung

pada karakteristik dataset, algoritma yang digunakan, dan tujuan pengendalian kualitas.

SMOTE menambah sampel sintetis pada kelas minoritas, sedangkan *class weight* menyesuaikan kontribusi kesalahan tiap kelas selama pelatihan. Keduanya dapat dibandingkan melalui evaluasi pada data validasi atau pengujian yang tetap mempertahankan distribusi kelas yang representatif. Dalam konteks deteksi cacat, recall kelas cacat, precision, F1-score, dan AUC-ROC dapat digunakan bersama-sama untuk menilai hasilnya.

BAB III
METODOLOGI

3.1 Alur Kerja (*Flowchart*)



3.2 Dataset

- **Nama:** Predicting Manufacturing Defects Dataset
- **Sumber:** https://www.kaggle.com/datasets/rabieelkharoua/predicting-manufacturing-defects-dataset
- **Jumlah data:** 3.240 entri

- **Jumlah fitur:** 16 kolom
- **Target:** DefectStatus (0 = Low Defect, 1 = High Defect)
- **Deskripsi kolom:** (tabel)

3.3 Tools dan Library

- Anaconda (Python 3.11)
- Jupyter Notebook
- pandas, numpy, scikit-learn, matplotlib, seaborn, imbalanced-learn

BAB IV
HASIL DAN PEMBAHASAN

4.1 Eksplorasi Data (EDA)

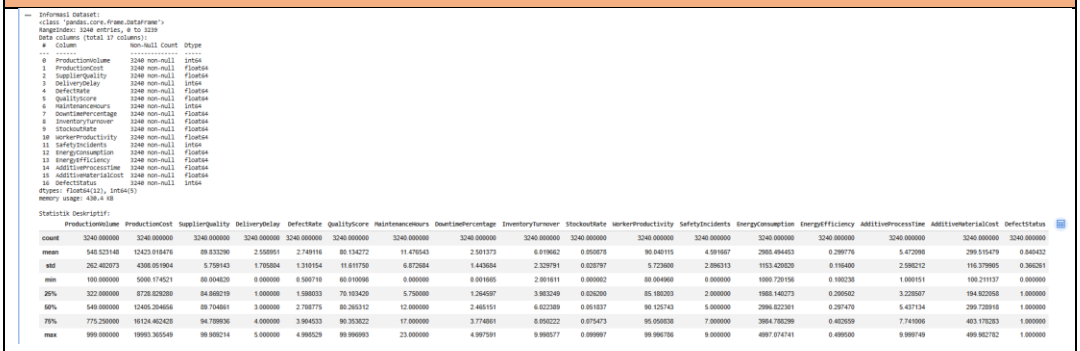
4.1.1 Informasi Umum Dataset

Kode

```
# Informasi tipe data dan missing value
print("Informasi Dataset:")
df.info()

# Statistik deskriptif
print("\nStatistik Deskriptif:")
df.describe()
```

Output



```
Informasi Dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3240 entries, 0 to 3239
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   ProductionVolume      3240 non-null   int64
 1   ProductionCost        3240 non-null   float64
 2   SupplierQuality       3240 non-null   float64
 3   DeliveryDelay         3240 non-null   int64
 4   DefectRate            3240 non-null   float64
 5   QualityScore          3240 non-null   float64
 6   MaintenanceHours      3240 non-null   int64
 7   DowntimePercentage    3240 non-null   float64
 8   InventoryTurnover     3240 non-null   float64
 9   StockoutRate          3240 non-null   float64
 10  WorkerProductivity    3240 non-null   float64
 11  SafetyIncidents       3240 non-null   int64
 12  EnergyConsumption     3240 non-null   float64
 13  EnergyEfficiency      3240 non-null   float64
 14  AdditiveProcessTime   3240 non-null   float64
 15  AdditiveMaterialCost  3240 non-null   float64
 16  DefectStatus          3240 non-null   int64
dtypes: float64(12), int64(5)
memory usage: 430.4 KB

Statistik Deskriptif:
```

	ProductionVolume	ProductionCost	SupplierQuality	DeliveryDelay	DefectRate	QualityScore	MaintenanceHours	DowntimePercentage	InventoryTurnover	StockoutRate	WorkerProductivity	SafetyIncidents	EnergyConsumption	EnergyEfficiency	AdditiveProcessTime	AdditiveMaterialCost	DefectStatus
count	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000	3240.000000
mean	548.523148	12423.018476	89.833290	2.558951	2.749116	80.134272	11.476543	2.501373	6.019662	0.050878	90.040115	4.591667	2988.494453	0.299776	5.472098	299.515479	0.840432
std	262.402073	4308.051904	5.759143	1.705804	1.310154	11.611750	6.872684	1.443684	2.329791	0.028797	5.723600	2.896313	1153.420620	0.116400	2.598212	116.379905	0.366261
min	100.000000	5000.174521	80.004620	0.000000	0.500710	60.010098	0.000000	0.001605	2.001611	0.000002	80.004960	0.000000	1000.720156	0.100238	1.000151	100.211137	0.000000
25%	322.000000	8728.829280	84.869219	1.000000	1.598033	70.103420	5.750000	1.264597	3.983249	0.026200	85.180203	2.000000	1988.140273	0.200502	3.228507	194.922058	1.000000
50%	549.000000	12405.204656	89.704861	3.000000	2.708775	80.265312	12.000000	2.465151	6.022389	0.051837	90.125743	5.000000	2996.822301	0.297470	5.437134	299.728918	1.000000
75%	775.250000	16124.462428	94.789936	4.000000	3.904533	90.353622	17.000000	3.774861	8.050222	0.075473	95.050838	7.000000	3984.788299	0.402659	7.741006	403.178283	1.000000
max	999.000000	19963.365549	99.989214	5.000000	4.998529	99.996993	23.000000	4.997591	9.998577	0.099997	99.996786	9.000000	4997.074741	0.499500	9.999749	499.982782	1.000000

Penjelasan:

- `df.info()` menampilkan tipe data dan jumlah nilai non-null per kolom.
- `df.describe()` memberikan statistik ringkasan (mean, std, min, max, quartile) untuk kolom numerik

4.1.2 Pengecakan *Missing Value* dan Duplikat

Kode

```
# Cek missing value
print("Jumlah Missing Value per Kolom:")
missing = df.isnull().sum()
print(missing[missing > 0] if any(missing > 0) else "Tidak ada missing value!")

# Cek duplikat
print(f"\nJumlah Data Duplikat: {df.duplicated().sum()}")
```

Output

```
Jumlah Missing Value per Kolom:
Tidak ada missing value!

Jumlah Data Duplikat: 0
```

Penjelasan:

- `df.isnull().sum()` menghitung berapa banyak nilai kosong di setiap kolom.
- `df.duplicated().sum()` menghitung baris yang persis sama (duplikat).

4.1.3 Distribusi Target

Kode

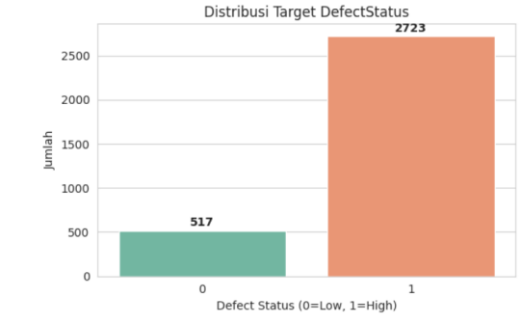
```
# Distribusi target
print("Distribusi Target (DefectStatus):")
print(df['DefectStatus'].value_counts())
print(f"\nPersentase:")
print(df['DefectStatus'].value_counts(normalize=True) * 100)

# Visualisasi
plt.figure(figsize=(6, 4))
sns.countplot(x='DefectStatus', data=df, palette='Set2')
plt.title('Distribusi Target DefectStatus')
plt.xlabel('Defect Status (0=Low, 1=High)')
plt.ylabel('Jumlah')
for i, v in enumerate(df['DefectStatus'].value_counts().sort_index()):
    plt.text(i, v + 50, str(v), ha='center', fontweight='bold')
plt.tight_layout()
plt.show()
```

Output

```
Distribusi Target (DefectStatus):
DefectStatus
1    2723
0     517
Name: count, dtype: int64

Persentase:
DefectStatus
1    84.04321
0    15.95679
Name: proportion, dtype: float64
```



Penjelasan:

- Kode tersebut digunakan untuk mengetahui distribusi kelas pada variabel target DefectStatus.
- `value_counts()` menghitung jumlah data pada setiap kelas.
- `value_counts(normalize=True) * 100` menghitung persentase masing-masing kelas.
- `sns.countplot()` digunakan untuk memvisualisasikan distribusi kelas dalam bentuk diagram batang.
- `plt.text()` menampilkan jumlah data di atas setiap batang.
- `plt.tight_layout()` mengatur tata letak grafik agar lebih rapi, dan plt.show() menampilkan hasil visualisasi.
- Visualisasi ini bertujuan untuk mengetahui perbandingan jumlah produk dengan status Low Defect (0) dan High Defect (1)

4.1.4 Visualisasi Distribusi Fitur Numerik

Kode

```
# Pilih fitur numerik (semua kecuali target)
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
numeric_cols.remove('DefectStatus')

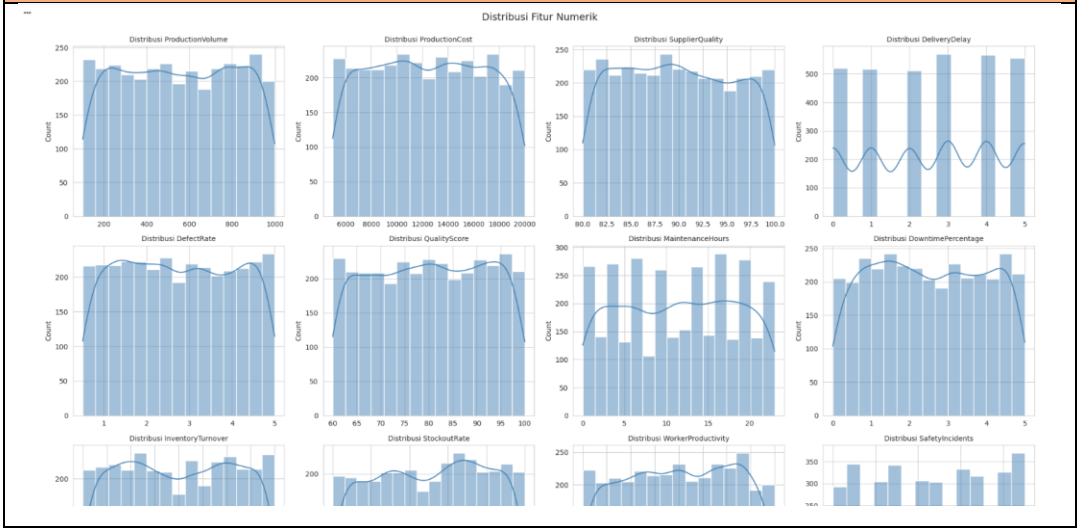
# Histogram distribusi
fig, axes = plt.subplots(4, 4, figsize=(20, 16))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, ax=axes[i], color='steelblue')
    axes[i].set_title(f'Distribusi {col}', fontsize=10)
    axes[i].set_xlabel('')

# Hapus subplot kosong
for j in range(len(numeric_cols), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.suptitle('Distribusi Fitur Numerik', fontsize=14, y=1.02)
plt.show()
```

Output



Penjelasan:

Cell ini menghasilkan histogram untuk setiap fitur numerik selain DefectStatus, disertai kurva KDE (Kernel Density Estimation). Grafik memperlihatkan pola persebaran nilai pada fitur seperti volume produksi, biaya produksi, kualitas pemasok, tingkat cacat, skor kualitas, jam pemeliharaan, dan konsumsi energi.

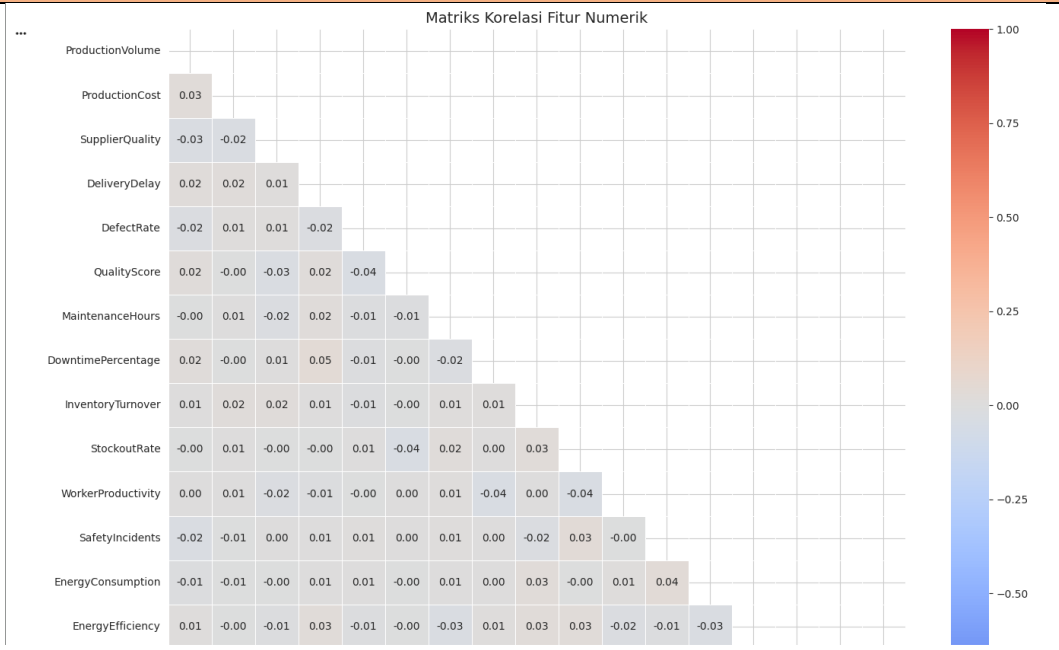
Histogram digunakan untuk melihat rentang nilai, konsentrasi data, dan bentuk distribusi setiap fitur. Sebagian fitur tampak memiliki persebaran yang relatif merata pada rentang tertentu, sedangkan fitur diskret seperti DeliveryDelay menunjukkan kelompok nilai yang terpisah. Visualisasi ini membantu mengidentifikasi karakteristik data serta menentukan kebutuhan prapemrosesan sebelum pemodelan.

4.1.5 Heatmap Korelasi

Kode

```
# Heatmap korelasi fitur numerik
plt.figure(figsize=(14, 12))
correlation = df[numeric_cols + ['DefectStatus']].corr()
mask = np.triu(np.ones_like(correlation, dtype=bool))
sns.heatmap(correlation, mask=mask, annot=True, cmap='coolwarm',
            fmt='.2f', linewidths=0.5, vmin=-1, vmax=1)
plt.title('Matriks Korelasi Fitur Numerik', fontsize=14)
plt.tight_layout()
plt.show()
```

Output



Penjelasan:

Pada hasil ini, korelasi beberapa fitur dengan DefectStatus terlihat relatif lebih besar dibandingkan fitur lainnya, terutama MaintenanceHours, DefectRate, dan QualityScore. Secara visual, MaintenanceHours dan DefectRate memiliki korelasi positif sekitar 0,30 dan 0,25, sedangkan QualityScore memiliki korelasi negatif sekitar -0,20. Sebagian besar fitur lain menunjukkan korelasi yang relatif lemah terhadap target.

4.1.6 Analisis Fitur vs Target

Kode

```
# Boxplot fitur vs target
fig, axes = plt.subplots(4, 4, figsize=(20, 16))
axes = axes.flatten()

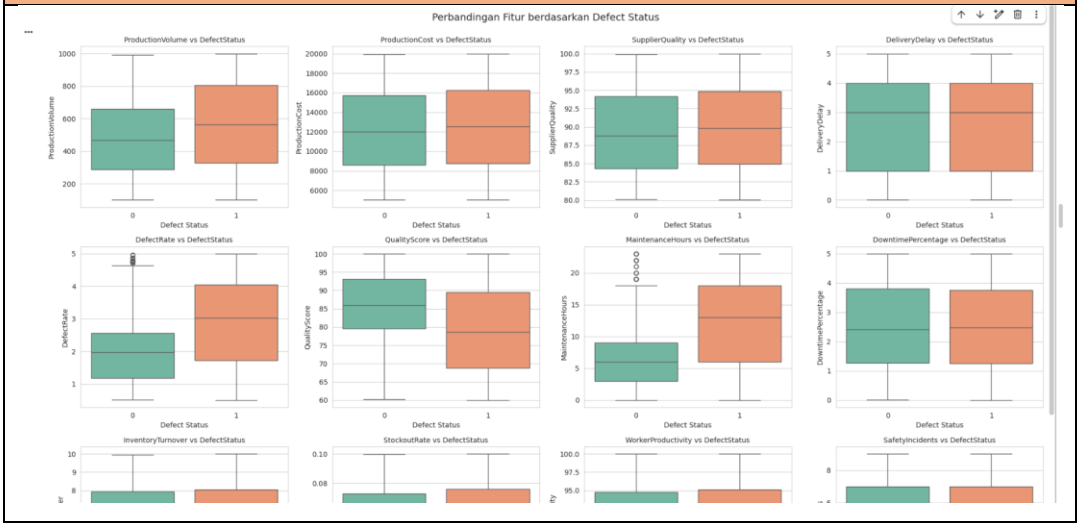
for i, col in enumerate(numeric_cols):
    sns.boxplot(x='DefectStatus', y=col, data=df, ax=axes[i],
palette='Set2')
    axes[i].set_title(f'{col} vs DefectStatus', fontsize=10)
    axes[i].set_xlabel('Defect Status')

for j in range(len(numeric_cols), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
```

```
plt.suptitle('Perbandingan Fitur berdasarkan Defect Status',
fontsize=14, y=1.02)
plt.show()
```

Output



Penjelasan:

Cell ini menghasilkan boxplot untuk membandingkan distribusi setiap fitur numerik pada dua kelompok, yaitu Low Defect dan High Defect. Boxplot menampilkan median, rentang kuartil, serta kemungkinan nilai pencilan (outlier) pada masing-masing kelompok.

Visualisasi ini digunakan untuk melihat apakah suatu fitur memiliki perbedaan distribusi antara kedua kelas. Perbedaan median atau rentang data dapat mengindikasikan bahwa fitur tersebut berpotensi membantu proses klasifikasi. Namun, apabila kedua boxplot saling tumpang tindih, fitur tersebut belum tentu dapat memisahkan kelas dengan jelas jika digunakan secara individual.

4.2 Preprocessing

4.2.1 Feature Engineering

Kode

```
# Fitur baru: total cost
df['Total_Cost'] = df['ProductionVolume'] * df['ProductionCost']

# Fitur baru: efficiency ratio
df['Efficiency_Ratio'] = df['WorkerProductivity'] /
(df['EnergyConsumption'] + 1)
```

```
# Fitur baru: quality index
df['Quality_Index'] = df['QualityScore'] / (df['DefectRate'] + 1)

# Fitur baru: maintenance intensity
df['Maintenance_Intensity'] = df['MaintenanceHours'] /
(df['DowntimePercentage'] + 1)

print("Fitur baru yang dibuat:")
print(df[['Total_Cost', 'Efficiency_Ratio', 'Quality_Index',
'Maintenance_Intensity']].describe())
```

Output

```
Fitur baru yang dibuat:
         Total_Cost  Efficiency_Ratio  Quality_Index  Maintenance_Intensity
count  3.240000e+03       3240.000000    3240.000000            3240.000000
mean   6.847746e+06          0.036326      24.775336               4.129230
std    4.262288e+06          0.017744      11.078886               3.628246
min    5.300488e+05          0.016262      10.283428               0.000000
25%    3.442429e+06          0.022630      16.263507               1.579725
50%    5.930295e+06          0.030356      21.522663               3.298635
75%    9.622630e+06          0.045006      30.520123               5.418093
max    1.969984e+07          0.096917      65.665111              22.351118
```

Penjelasan:

Cell ini membuat empat fitur baru dari fitur yang telah tersedia:

1. Total_Cost = ProductionVolume × ProductionCost.
2. Efficiency_Ratio = WorkerProductivity / (EnergyConsumption + 1).
3. Quality_Index = QualityScore / (DefectRate + 1).
4. Maintenance_Intensity = MaintenanceHours / (DowntimePercentage + 1).

Output statistik deskriptif menunjukkan bahwa seluruh fitur baru memiliki 3.240 nilai. Rata-rata Total_Cost sekitar 6.847.746, Efficiency_Ratio sekitar 0,0363, Quality_Index sekitar 24,7753, dan Maintenance_Intensity sekitar 4,1292.

4.2.2 *Split Data*

Kode

```
# Bagi data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Jumlah data training:", X_train.shape[0])
print("Jumlah data testing:", X_test.shape[0])
print("\nDistribusi target di training:")
print(y_train.value_counts(normalize=True))
```

```
print("\nDistribusi target di testing:")
print(y_test.value_counts(normalize=True))
```

Output

```
Jumlah data training: 2592
Jumlah data testing: 648

Distribusi target di training:
DefectStatus
1    0.840278
0    0.159722
Name: proportion, dtype: float64

Distribusi target di testing:
DefectStatus
1    0.841049
0    0.158951
Name: proportion, dtype: float64
```

Penjelasan:

Output menunjukkan bahwa dataset dibagi menjadi:

- Data training: 2.592 observasi atau 80%.
- Data testing: 648 observasi atau 20%.

Parameter stratify=y digunakan agar proporsi kelas target pada data pelatihan dan pengujian tetap mendekati proporsi awal. Hasilnya, kelas 1 memiliki proporsi sekitar 84,03% pada data training dan 84,10% pada data testing, sedangkan kelas 0 masing-masing sekitar 15,97% dan 15,90%.

Pembagian ini bertujuan melatih model menggunakan data training dan menilai kemampuan generalisasinya menggunakan data testing yang tidak digunakan dalam pelatihan.

4.2.3 *Feature Scaling*

Kode

```
# Standardisasi fitur numerik
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Mean training setelah scaling (harus mendekati 0):")
print(X_train_scaled.mean(axis=0).round(4)[:5], "...")
print("\nStd training setelah scaling (harus mendekati 1):")
print(X_train_scaled.std(axis=0).round(4)[:5], "...")
```

Output

```
...  Mean training setelah scaling (harus mendekati 0):
     [ 0. -0. -0. -0.  0.] ...

     Std training setelah scaling (harus mendekati 1):
     [1. 1. 1. 1. 1.] ...
```

Penjelasan:

Output menunjukkan bahwa rata-rata lima fitur pertama setelah scaling mendekati 0 dan simpangan bakunya mendekati 1.

Cell ini menggunakan StandardScaler untuk melakukan standardisasi fitur. Parameter standardisasi dipelajari melalui fit_transform() pada data training, kemudian diterapkan pada data testing menggunakan transform(). Proses ini mengubah skala fitur agar memiliki rata-rata mendekati nol dan simpangan baku mendekati satu berdasarkan data pelatihan.

Standardisasi sangat berguna bagi algoritma yang sensitif terhadap skala fitur, seperti KNN dan Logistic Regression. Pada Decision Tree dan Random Forest, standardisasi umumnya tidak menjadi persyaratan utama, tetapi seluruh model dalam notebook ini menggunakan data yang telah distandardisasi agar proses pemodelan konsisten

4.3 Modeling

Kode

```
# # Inisialisasi model
models = {
    'Decision Tree': DecisionTreeClassifier(max_depth=5,
random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'Naive Bayes': GaussianNB(),
    'Logistic Regression': LogisticRegression(max_iter=1000,
random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100,
max_depth=10, random_state=42)
}

# Dictionary untuk menyimpan hasil
results = {}

# Latih dan evaluasi setiap model
for name, model in models.items():
    # Latih model
```

```
    model.fit(X_train_scaled, y_train)

    # Prediksi
    y_train_pred = model.predict(X_train_scaled)
    y_test_pred = model.predict(X_test_scaled)
    y_test_proba = model.predict_proba(X_test_scaled)[:, 1]

    # Hitung metrik
    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)
    precision = precision_score(y_test, y_test_pred, zero_division=0)
    recall = recall_score(y_test, y_test_pred, zero_division=0)
    f1 = f1_score(y_test, y_test_pred, zero_division=0)
    auc = roc_auc_score(y_test, y_test_proba)

    # Simpan hasil
    results[name] = {
        'model': model,
        'train_accuracy': train_acc,
        'test_accuracy': test_acc,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'auc_roc': auc,
        'y_test_pred': y_test_pred,
        'y_test_proba': y_test_proba
    }

    print(f"{'='*60}")
    print(f"Model: {name}")
    print(f"Train Accuracy : {train_acc:.4f}")
    print(f"Test Accuracy  : {test_acc:.4f}")
    print(f"Precision      : {precision:.4f}")
    print(f"Recall         : {recall:.4f}")
    print(f"F1-Score       : {f1:.4f}")
    print(f"AUC-ROC        : {auc:.4f}")
print()
```

Output

```
============================================================
Model: Decision Tree
Train Accuracy : 0.9684
Test Accuracy  : 0.9383
Precision      : 0.9501
Recall         : 0.9780
F1-Score       : 0.9638
AUC-ROC        : 0.8618

============================================================
Model: K-Nearest Neighbors
Train Accuracy : 0.9132
Test Accuracy  : 0.8796
Precision      : 0.9005
Recall         : 0.9633
F1-Score       : 0.9309
AUC-ROC        : 0.7484

============================================================
Model: Naive Bayes
Train Accuracy : 0.8877
Test Accuracy  : 0.8704
Precision      : 0.9229
Recall         : 0.9229
F1-Score       : 0.9229
AUC-ROC        : 0.7973

============================================================
Model: Logistic Regression
Train Accuracy : 0.8808
Test Accuracy  : 0.8796
Precision      : 0.8938
Recall         : 0.9725
F1-Score       : 0.9315
AUC-ROC        : 0.7991

============================================================
Model: Random Forest
Train Accuracy : 0.9803
Test Accuracy  : 0.9506
Precision      : 0.9508
Recall         : 0.9927
F1-Score       : 0.9713
AUC-ROC        : 0.8163
```

Penjelasan:

Berdasarkan hasil tersebut, Random Forest memperoleh test accuracy tertinggi sebesar 95,06%, recall sebesar 99,27%, dan F1-score sebesar 97,13%. Hasil ini menunjukkan bahwa Random Forest memiliki kinerja terbaik di antara kelima model berdasarkan F1-score dan mampu mengenali sebagian besar data kelas positif, yaitu DefectStatus = 1.

Sementara itu, Decision Tree memiliki nilai AUC-ROC tertinggi sebesar 0,8618. Hal ini menunjukkan bahwa kemampuan membedakan kedua kelas berdasarkan skor prediksi tidak selalu sejalan dengan peringkat model berdasarkan accuracy atau F1-score. Pemilihan model akhir perlu disesuaikan dengan metrik yang paling relevan terhadap kebutuhan deteksi cacat.

4.4 Evaluasi Model

4.4.1 Tabel Perbandingan Metrik

Kode

```
# Buat DataFrame perbandingan
comparison_df = pd.DataFrame({
    'Model': list(results.keys()),
    'Train Acc': [results[m]['train_accuracy'] for m in results],
    'Test Acc': [results[m]['test_accuracy'] for m in results],
    'Precision': [results[m]['precision'] for m in results],
    'Recall': [results[m]['recall'] for m in results],
    'F1-Score': [results[m]['f1_score'] for m in results],
    'AUC-ROC': [results[m]['auc_roc'] for m in results]
}).round(4)

# Hitung selisih (indikasi overfitting)
comparison_df['Overfit Gap'] = (
    comparison_df['Train Acc'] - comparison_df['Test Acc']
).round(4)

comparison_df = comparison_df.sort_values('F1-Score', ascending=False)
print("Perbandingan Model:")
print(comparison_df.to_string(index=False))
```

Model	Train Accuracy	Test Accuracy	Precision	Recall	1-Score	UC-ROC
Decision Tree	0,9684	0,9383	0,9501	0,9780	0,9638	0,8618
KNN	0,9132	0,8796	0,9005	0,9633	0,9309	0,7484
Naive Bayes	0,8877	0,8704	0,9229	0,9229	0,9229	0,7973
Logistic Regression	0,8808	0,8796	0,8938	0,9725	0,9315	0,7991
Random Forest	0,9803	0,9506	0,9508	0,9927	0,9713	0,8163

Penjelasan:

Output menunjukkan bahwa Random Forest menempati peringkat pertama dengan F1-score 0,9713, diikuti Decision Tree sebesar 0,9638. Logistic Regression, KNN, dan Naive Bayes memiliki F1-score masing-masing 0,9315, 0,9309, dan 0,9229.

Kolom Overfit Gap menunjukkan selisih antara akurasi training dan testing. Random Forest memiliki selisih 0,0297, Decision Tree 0,0301, Logistic Regression 0,0012, KNN 0,0336, dan Naive Bayes 0,0173. Selisih yang lebih besar dapat

menjadi indikasi awal bahwa model lebih sesuai dengan data training dibandingkan data testing. Namun, selisih tersebut bukan satu-satunya ukuran overfitting dan perlu dipertimbangkan bersama hasil validasi serta metrik lainnya

4.4.2 Confusion Matrix Model Terbaik

Kode

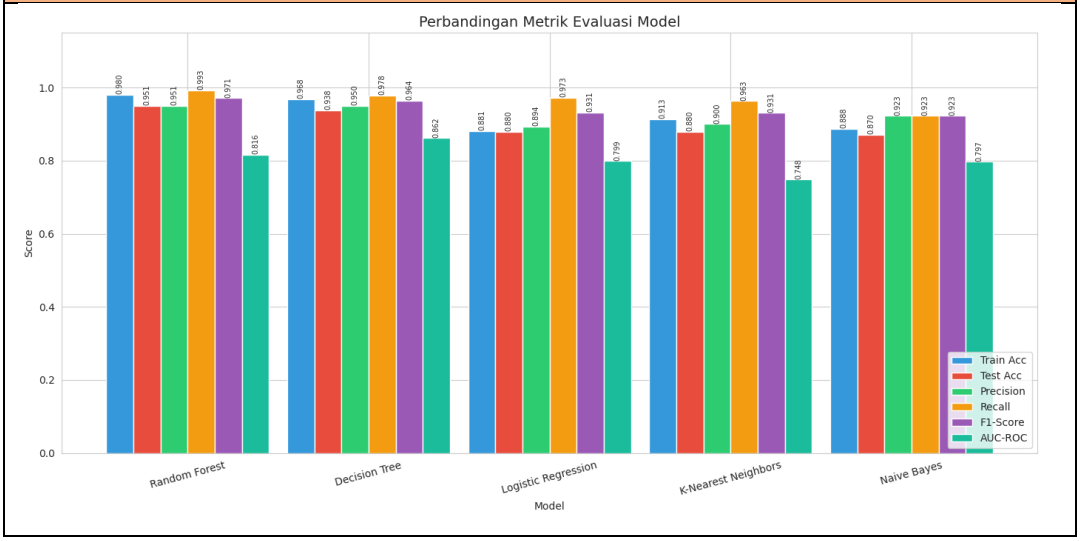
```
# Bar chart perbandingan
fig, ax = plt.subplots(figsize=(14, 7))
x = np.arange(len(comparison_df))
width = 0.15

metrics = ['Train Acc', 'Test Acc', 'Precision', 'Recall', 'F1-Score',
'AUC-ROC']
colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6',
'#1abc9c']

for i, (metric, color) in enumerate(zip(metrics, colors)):
    bars = ax.bar(x + i*width - 2.5*width, comparison_df[metric],
                  width, label=metric, color=color)
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.3f}', xy=(bar.get_x() +
bar.get_width()/2, height),
                    xytext=(0, 2), textcoords="offset points",
                    ha='center', fontsize=7, rotation=90)

ax.set_xlabel('Model')
ax.set_ylabel('Score')
ax.set_title('Perbandingan Metrik Evaluasi Model', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels(comparison_df['Model'], rotation=15)
ax.legend(loc='lower right')
ax.set_ylim(0, 1.15)
plt.tight_layout()
plt.show()
```

Output



Penjelasan:

Diagram batang membandingkan train accuracy, test accuracy, precision, recall, F1-score, dan AUC-ROC dari kelima model.

Secara visual, Random Forest memiliki nilai tinggi pada sebagian besar metrik, terutama recall dan F1-score. Decision Tree menunjukkan AUC-ROC yang lebih tinggi daripada model lainnya. Perbedaan ini menegaskan bahwa setiap metrik mengukur aspek kinerja yang berbeda.

Visualisasi ini mempermudah perbandingan kinerja model secara menyeluruh. Model terbaik tidak seharusnya dipilih hanya berdasarkan satu metrik tanpa mempertimbangkan tujuan klasifikasi, terutama karena dataset memiliki distribusi kelas yang tidak seimbang.

4.4.3 Classification Report

4.4.1 Kode

```
# Ambil model terbaik berdasarkan F1-Score
best_model_name = comparison_df.iloc[0]['Model']
best_model = results[best_model_name]['model']
best_y_pred = results[best_model_name]['y_test_pred']

print(f"Model Terbaik: {best_model_name}\n")

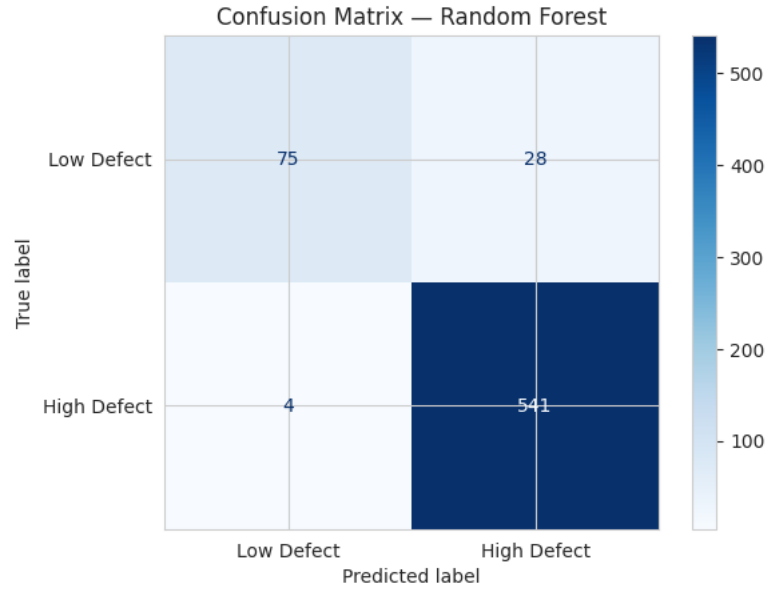
# Confusion Matrix
cm = confusion_matrix(y_test, best_y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
                                    display_labels=['Low Defect', 'High
Defect'])
disp.plot(cmap='Blues')
plt.title(f'Confusion Matrix — {best_model_name}')
plt.show()

# Classification Report
print(f"Classification Report — {best_model_name}:\n")
print(classification_report(y_test, best_y_pred,
                            target_names=['Low Defect', 'High
Defect']))
```

Output

```
Model Terbaik: Random Forest
```




```
Classification Report — Random Forest:

              precision    recall  f1-score   support

  Low Defect       0.95      0.73      0.82       103
 High Defect       0.95      0.99      0.97       545

    accuracy                           0.95       648
   macro avg       0.95      0.86      0.90       648
weighted avg       0.95      0.95      0.95       648
```

Penjelasan:

Dari 648 data testing, model mengklasifikasikan 75 data Low Defect dan 541 data High Defect dengan benar. Sebanyak 28 data Low Defect salah diprediksi sebagai High Defect, sedangkan 4 data High Defect salah diprediksi sebagai Low Defect.

Classification report menunjukkan bahwa kelas Low Defect memiliki precision 0,95, recall 0,73, dan F1-score 0,82. Sementara itu, kelas High Defect memiliki precision 0,95, recall 0,99, dan F1-score 0,97. Akurasi keseluruhan model adalah 0,95 atau sekitar 95%.

Hasil tersebut menunjukkan bahwa Random Forest sangat baik dalam mendeteksi kelas High Defect, tetapi masih melewatkan sebagian data Low Defect karena kelas tersebut hanya berhasil dikenali dengan recall sekitar 73%. Oleh sebab itu, interpretasi hasil harus memperhatikan arti setiap kelas dan tujuan penggunaan model, bukan hanya akurasi keseluruhan.

4.4.2 ROC Curve

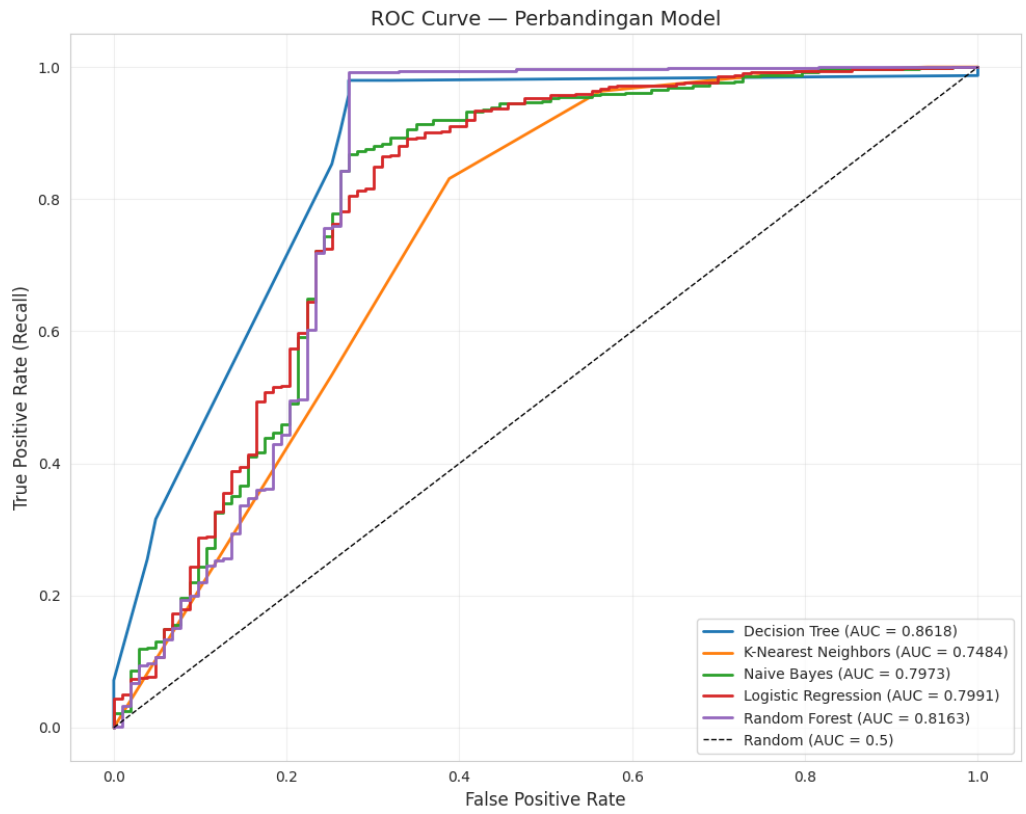
4.4.1 Kode

```
# ROC Curve untuk semua model
plt.figure(figsize=(10, 8))

for name in results:
    y_proba = results[name]['y_test_proba']
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    auc = results[name]['auc_roc']
    plt.plot(fpr, tpr, linewidth=2, label=f'{name} (AUC = {auc:.4f})')

plt.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random (AUC =
0.5)')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate (Recall)', fontsize=12)
plt.title('ROC Curve — Perbandingan Model', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Output



Penjelasan:

Cell ini menampilkan kurva ROC (Receiver Operating Characteristic) untuk kelima model berdasarkan nilai probabilitas prediksi. Sumbu horizontal menunjukkan False Positive Rate, sedangkan sumbu vertikal menunjukkan True Positive Rate atau Recall. Garis diagonal putus-putus menunjukkan acuan prediksi acak dengan AUC sebesar 0,5. Nilai AUC-ROC yang diperoleh adalah:

- Decision Tree: 0,8618.
- Random Forest: 0,8163.
- Logistic Regression: 0,7991.
- Naive Bayes: 0,7973.
- KNN: 0,7484.

Decision Tree memiliki AUC-ROC tertinggi, yaitu 0,8618, sehingga menunjukkan kemampuan diskriminasi terbaik di antara model yang diuji berdasarkan metrik ini. Random Forest memiliki F1-score tertinggi, tetapi AUC-ROC-nya lebih rendah daripada Decision Tree. Perbedaan ini menunjukkan bahwa

kemampuan membedakan kelas berdasarkan skor probabilitas dan kemampuan menghasilkan prediksi kelas pada ambang tertentu merupakan aspek yang berbeda.

4.5 Pembahasan

4.5.1 Analisis Model Terbaik

Berdasarkan hasil evaluasi, **Random Forest** merupakan model terbaik berdasarkan *F1-score* sebesar 97,13%, *accuracy* 95,06%, dan *recall* 99,27%. Model ini mampu mempelajari pola kompleks melalui penggabungan beberapa pohon keputusan. Hasil *cross-validation* juga menunjukkan rata-rata *F1-score* sebesar 97,53%, sehingga Random Forest menjadi kandidat yang baik untuk klasifikasi cacat produk manufaktur.

4.5.2 Analisis Overfitting

Random Forest memiliki *accuracy* training sebesar 98,03% dan testing sebesar 95,06%, dengan selisih 2,97%. Selisih tersebut menunjukkan kemungkinan perbedaan kinerja pada data training dan testing, tetapi belum membuktikan adanya *overfitting* yang serius. Upaya pencegahan dapat dilakukan dengan membatasi kedalaman pohon, mengatur *hyperparameter*, dan melakukan validasi silang.

4.5.3 Fitur Paling Berpengaruh

Berdasarkan *feature importance*, fitur paling berpengaruh adalah `MaintenanceHours` (0,1601), diikuti `QualityScore` (0,1404), `DefectRate` (0,1353), `Quality_Index` (0,1323), dan `Maintenance_Intensity` (0,0887). Fitur-fitur tersebut memberikan kontribusi relatif besar terhadap prediksi model. Namun, nilai *importance* tidak membuktikan hubungan sebab-akibat.

4.5.4 Implikasi untuk Quality Control

Model Random Forest berpotensi membantu mengidentifikasi produk yang berisiko mengalami cacat sehingga dapat diprioritaskan untuk pemeriksaan lebih lanjut. Fitur yang memiliki *importance* tinggi juga dapat menjadi bahan analisis bagi tim produksi. Sebelum diterapkan, model perlu diuji menggunakan data produksi terbaru dan hasil prediksinya tetap perlu didukung prosedur inspeksi kualitas.

4.5.5 Keterbatasan

Praktikum ini memiliki beberapa keterbatasan, yaitu penggunaan satu dataset dengan distribusi kelas yang tidak seimbang, fitur hasil *feature engineering* yang belum diuji secara terpisah, serta keterbatasan *feature importance* dalam menjelaskan hubungan sebab-akibat. Selain itu, model belum diuji secara langsung pada lini produksi. Oleh karena itu, diperlukan pengujian lebih lanjut menggunakan data baru dan prapemrosesan yang tepat untuk memastikan keandalan model.

BAB V
HASIL DAN PEMBAHASAN

5.1 Kesimpulan

Berdasarkan hasil praktikum klasifikasi cacat produk pada data manufaktur, dapat disimpulkan bahwa:

1. Dataset terdiri dari 3.240 data dengan 17 fitur awal dan 4 fitur tambahan hasil *feature engineering*. Distribusi kelas tidak seimbang, yaitu 84,04% kelas *High Defect* dan 15,96% kelas *Low Defect*.
2. Lima algoritma klasifikasi telah diuji, yaitu Decision Tree, K-Nearest Neighbors (KNN), Naive Bayes, Logistic Regression, dan Random Forest.
3. **Random Forest menjadi model terbaik berdasarkan F1-score sebesar 97,13%**, dengan *accuracy* 95,06% dan *recall* 99,27% pada data pengujian.
4. Fitur yang paling berpengaruh terhadap prediksi adalah MaintenanceHours, diikuti QualityScore dan DefectRate.
5. Model berpotensi membantu proses *quality control* dalam mengidentifikasi risiko cacat produk, tetapi perlu diuji lebih lanjut sebelum diterapkan pada lini produksi.

5.2 Saran

Untuk pengembangan selanjutnya, beberapa saran yang dapat dilakukan adalah:

1. Menambah jumlah dan variasi data agar model dapat mempelajari kondisi produksi yang lebih beragam.
2. Mencoba algoritma lain, seperti XGBoost, serta melakukan optimasi *hyperparameter* untuk meningkatkan performa model.
3. Menangani ketidakseimbangan kelas dengan metode seperti SMOTE atau class_weight, sesuai tujuan klasifikasi dan kelas yang ingin ditingkatkan deteksinya.
4. Melakukan validasi silang dengan *pipeline* prapemrosesan yang tepat untuk mengurangi risiko kebocoran data.
5. Menguji model menggunakan data produksi terbaru dan memantau kinerjanya secara berkala sebelum digunakan sebagai pendukung keputusan *quality control*.

Link Github Tugas mandiri:

https://github.com/miftahrizqi-coder/Machine-Learning-Practice.git